VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Belgaum-590018.



BIO INSPIRED ALGORITHMS (23DS7BSBIO) PROJECT REPORT

ON

SwarmFolio: Multi-Objective Stock Portfolio Optimization Using Particle Swarm Optimization and Genetic Algorithms

Risk, return and tail-risk trade-offs with a live optimisation workbench

Submitted in Partial fulfilment for the VII Semester, B.E in Artificial Intelligence & Data Science

Submitted by:

Vivek Boora	**1BM23AD073**
Ayush Pandey	**1BM23AD074**
Nitin	**1BM24AD404**

Under the Guidance of:

Prof. Suman M
Assistant Professor
B.E, M.Tech, [PhD]
Dept. of AI & DS



2026-2027

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

B.M.S. College of Engineering

Bull Temple Road, Basavanagudi, Bengaluru-560019

(Autonomous Institute under Visvesvaraya Technological University, Belagavi and Approved by AICTE, New Delhi, NAAC Accredited with 'A++' Grade)

B.M.S. College of Engineering

Bull Temple Road, Basavanagudi, Bengaluru-560019

(Autonomous Institute under Visvesvaraya Technological University, Belagavi and Approved by AICTE, New Delhi, NAAC Accredited with 'A++' Grade)

Department of Artificial Intelligence & Data Science

CERTIFICATE

Certified that the BIO-INSPIRED ALGORITHMS [23DS7BSBIO] entitled "**SwarmFolio: Multi-Objective Stock Portfolio Optimization Using Particle Swarm Optimization and Genetic Algorithms**" carried out by **Vivek Boora, Ayush Pandey and Nitin** bonafide students of B.M.S. College of Engineering, in partial fulfillment for the award of Bachelor of Engineering in Artificial Intelligence & Data Science of the Visvesvaraya Technological University, Belgaum during the year 2026-27. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report deposited in the department library.

Signature of the Guide

Prof. Suman M
Asst. Prof. Dept. of AI & DS
BMSCE, Bengaluru.

Signature of the HOD

Dr. Seemanthini K
Assoc Prof. & Head, Dept. of AI & DS
BMSCE, Bengaluru.

Principal
Dr. Bheemsha Arya
B.M.S. College of Engineering

External Viva

	Name of the Examiners	Signature with Date
1.		
2.		

ABSTRACT

Dividing money across stocks involves conflicting goals: high return, low volatility and limited losses in bad periods. This project, SwarmFolio, treats long-only portfolio selection as a three-objective problem (annual return, annual volatility and Conditional Value-at-Risk) with a cap on each holding, a minimum holding size and an optional limit on the number of holdings. A multi-objective Particle Swarm Optimizer and an NSGA-II genetic algorithm are built on one shared repair operator, so that both search the same feasible space and can be compared fairly. The system is delivered as a Python library, a command-line tool and a browser workbench that animates both populations converging on their Pareto fronts and lets the user inspect any portfolio on them.

Six experiments on seeded simulated markets show that both algorithms find much better fronts than random search at an equal budget (hypervolume about 0.70 against 0.25) and never violate a constraint. They are close to each other: the swarm converges in far fewer iterations and gives a more even front, while the genetic algorithm ends marginally ahead from about 100 iterations. In 30 independent markets, portfolios chosen for the best Sharpe ratio fell from about 1.34 in training to about 0.53 on held-out data, no better than equal weight (0.59), whereas minimum-volatility portfolios had lower held-out volatility than equal weight in all 30 markets. A good front is therefore no promise of future return, and risk-oriented choices from the front are the more dependable.

DECLARATION

We the undersigned students of 7th semester, Department of Artificial Intelligence & Data Science, B.M.S. College of Engineering, declare that the project work for bio-inspired algorithms [23DS7BSBIO] entitled "**SwarmFolio: Multi-Objective Stock Portfolio Optimization Using Particle Swarm Optimization and Genetic Algorithms**", is a bonafide work of us and our project is neither a copy nor by any means a modification of any other engineering project.

We also declare that this project was not entitled for submission to any other university in the past and shall remain the only submission made and will not be submitted by us to any other university in the future.

Name	**USN**	**Signature**
Vivek Boora	**1BM23AD073**	……
Ayush Pandey	**1BM23AD074**	……
Nitin	**1BM24AD404**	……

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crowned our effort with success.

We are grateful to our institution, **B.M.S. College of Engineering**, with its ideals and inspirations for having provided us with the facilities, which has made this project a success.

We earnestly thank **Dr. Bheemsha Arya, Principal, BMS College of Engineering**, for facilitating academic excellence in the college and providing us with the congenial environment to work in, that helped us in completing this project.

We wish to extend our profound thanks to **Dr. Seemanthini K, Assoc Prof. and Head, Department of Artificial Intelligence & Data Science, BMSCE**, for giving us the consent to carry out this project.

We would like to express our sincere thanks to our guide **Prof. Suman M, Assistant Professor, Department of Artificial Intelligence & Data Science, BMSCE**, for the valuable guidance at every stage, which helped us in the successful completion of the project.

We would like to thank all the teaching and non-teaching staff for their valuable advice and support.

We would like to express our sincere thanks to our parents and friends for their support.

Vivek Boora

Ayush Pandey

Nitin

TABLE OF CONTENTS

TITLE	PAGE NO.
ABSTRACT	**i**
DECLARATION	**ii**
ACKNOWLEDGEMENTS	**iii**
LIST OF TABLES	**vi**
LIST OF FIGURES	**vii**

CHAPTER NO.	TITLE	PAGE NO.
1	**Introduction**	**1**
1.1	Overview	1
1.2	Motivation	1
1.3	Objectives	1
1.4	Scope	2
1.5	Sustainable Development Goals (SDGs)	2
1.6	Existing System	2
1.7	Proposed System	3
1.8	Work Plan	3
2	**Literature Survey**	**4**
2.1	Overview	4
2.2	Detailed Survey	4
2.2.1	Foundations of portfolio selection	4
2.2.2	Constrained and multi-objective formulations	4
2.2.3	Evolutionary and swarm algorithms	4
2.2.4	Assessing the quality of a Pareto front	4
2.3	Research Gap Identification	5
2.4	Mapping of your Objective with Research Gap Identification	6
3	**Requirement Analysis and Specification**	**7**
3.1	Functional Requirements	7
3.2	Non-functional Requirements	7
3.3	Hardware Requirements	8
3.4	Software Requirements	8
3.5	Cost Estimation	8
3.6	Assumptions	8
3.7	Risks and Challenges	9
3.7.1	AI Security Challenges	9
4	**Design**	**10**

CHAPTER NO.	TITLE	PAGE NO.
4.1	High Level Design	10
4.1.1	System Architecture	10
4.1.2	Abstract specification of Sub-systems	11
4.1.3	Interface Design	11
4.2	Methodology	13
4.2.1	Research Design	13
4.2.2	Data Collection	13
5	**Implementation**	**15**
5.1	Pseudo code	15
5.1.1	Particle Swarm Optimization (PSO)	15
5.1.2	Genetic Algorithm (NSGA-II)	16
5.1.3	How the two algorithms differ	17
5.1.4	Pseudo code of the implementation	17
5.2	Use of appropriate advanced tools and techniques	19
5.3	Novelty	20
5.4	Mitigation of AI Security threats	20
6	**Testing and Validation**	**21**
6.1	Data interpretation	21
6.2	Analysis	21
6.3	Test results	21
6.3.1	Testing process	21
6.3.2	System test cases	22
7	**Results and Discussion**	**23**
7.1	Presentation of Results	23
7.1.1	Statistical Analysis	26
7.2	Figures	26
7.3	Subfigures	27
7.4	Discussion	28
7.5	Conclusion	30
8	**Conclusion**	**31**
8.1	Conclusion	31
8.2	Future Scope	31
	POs attained with Justification	32
	REFERENCES	33

LIST OF TABLES

Tables	Caption	Page No.
Table 1.1:	Relation of the project to the Sustainable Development Goals	2
Table 2.1:	Summary of the literature used in the project	5
Table 2.2:	Mapping of objectives to research gaps	6
Table 3.1:	Functional requirements	7
Table 3.2:	Non-functional requirements	7
Table 3.3:	Software requirements	8
Table 3.4:	Cost estimates	8
Table 3.5:	Risks and mitigations	9
Table 4.1:	Sub-systems and their responsibilities	11
Table 4.2:	HTTP interface of the server	12
Table 4.3:	Experimental design	13
Table 4.4:	Parameters of the synthetic market	14
Table 5.1:	Comparison of the two algorithms	17
Table 5.2:	Parameter settings used for both algorithms	17
Table 5.3:	Techniques and tools	19
Table 5.4:	Security threats and mitigations	20
Table 6.1:	System test cases and results	22
Table 7.1:	Front quality over 10 seeds (20 assets, 100 particles or individuals, 150 iterations)	23
Table 7.2:	Selected portfolios on the default market (seed 0)	23
Table 7.3:	Hypervolume against the holding limit (5 seeds each; 20 assets; scale shared within each row)	24
Table 7.4:	Hypervolume and run time against the number of assets (3 seeds; run time measured separately)	24
Table 7.5:	Hypervolume against budget (5 seeds each; one fixed reference for all rows)	25
Table 7.6:	Held-out performance over 30 independent markets (mean over markets; p is the paired Wilcoxon test of held-out Sharpe against equal weight)	25
Table 7.7:	Effect of removing an operator (8 seeds; limit of 8 holdings)	26
Table 7.8:	Summary of paired Wilcoxon signed-rank tests	26

LIST OF FIGURES

Figures	Caption	Page No.
Figure 1.1:	Work plan of the project (weeks of the semester).	3
Figure 4.1:	Three-layer architecture of SwarmFolio.	10
Figure 4.2:	Main screen after a run: control rail on the left, risk-return stage on the right.	12
Figure 4.3:	Portfolio inspector with selection controls, performance table and weights.	12
Figure 5.1:	Flowcharts of (a) the multi-objective particle swarm optimiser and (b) the NSGA-II genetic algorithm.	19
Figure 7.1:	Hypervolume against the number of objective evaluations (error bars: standard deviation over five seeds).	27
Figure 7.2:	Mean Sharpe ratio on the training and the held-out window for each strategy, over 30 markets.	27
Figure 7.3:	Fronts found on the training window of the default market: (a) return against volatility; (b) return against CVaR.	27
Figure 7.4:	Best training Sharpe on the front per iteration for seed 0 (a) and hypervolume over ten seeds (b).	28
Figure 7.5:	Best-Sharpe portfolios on the default market: (a) weights of the largest holdings; (b) held-out wealth against equal weight.	28
Figure 7.6:	Held-out distributions over 30 markets: (a) Sharpe ratio; (b) volatility; (c) maximum drawdown.	28

Chapter 1

Introduction

1.1 Overview

Portfolio selection is the problem of dividing a fixed amount of capital across a set of risky assets. The classical answer, mean-variance analysis [1], gives for each level of expected return the portfolio with the least variance; together these portfolios form the efficient frontier. Practice is harder than the textbook model. Investors care about losses in bad periods as well as variance, which Conditional Value-at-Risk (CVaR) measures [2]. Mandates also impose a cap on each holding, a minimum holding size and a limit on the number of holdings, and the last of these makes the problem hard for exact solvers [3]. Because the goals conflict, a good optimiser returns a set of non-dominated trade-offs and not a single answer.

Population-based metaheuristics suit this setting because one run returns a whole population. Particle Swarm Optimization (PSO) [4] and the NSGA-II genetic algorithm [5] are two well-known examples. This project, **SwarmFolio**, implements a multi-objective PSO (MOPSO) and NSGA-II on one shared portfolio model with three objectives (return, volatility, CVaR) and four constraints, and compares them with random search and the equal-weight portfolio on training and held-out data. It is delivered as a Python library with automated tests and as a browser workbench that animates both searches.

1.2 Motivation

Studies that compare PSO and GA for portfolios usually report one benchmark data set and in-sample results only. Michaud [6] argued that optimisers amplify estimation error, and DeMiguel et al. [7] found that the naive 1/N rule was hard to beat out of sample, so evaluation on unseen data and on many markets is needed. Comparisons are also confounded when each algorithm handles constraints differently; one shared repair operator removes that source of difference. Finally, seeing a population move toward the efficient frontier and then clicking a point to see its holdings makes dominance, diversity and over-fitting concrete for students.

1.3 Objectives

1. Formulate long-only portfolio selection with three objectives (return, volatility, CVaR) and cap, minimum-size and cardinality constraints.

2. Implement MOPSO and NSGA-II on one shared repair operator so that they can be compared fairly.
3. Measure front quality with hypervolume, spacing, front size and runtime, against random search at an equal budget.
4. Study the effect of tighter constraints, more assets, different budgets and removing individual operators.
5. Test on held-out data, over 30 independent markets, whether chosen portfolios beat the equal-weight portfolio.
6. Build a browser interface that runs both optimisers, animates the search and accepts the user's own prices.

1.4 Scope

In scope: long-only equity portfolios; daily returns; empirical estimates of return, volatility and CVaR; MOPSO and NSGA-II; random search and equal weight as baselines; a synthetic market generator and CSV import; a Python library, command-line tool, local web server and tests. **Out of scope:** live data and order execution, short selling, trading costs as an objective, return forecasting, and any claim of investment advice. The experiments use simulated markets; real-market validation is future work (Chapter 8).

1.5 Sustainable Development Goals (SDGs)

The project relates to the United Nations Sustainable Development Goals [8] in limited ways.

Table 1.1: Relation of the project to the Sustainable Development Goals

SDG	Relevance to this project
SDG 4: Quality Education	The workbench visualises dominance, Pareto fronts and over-fitting, and the code is open, so it can be used for teaching.
SDG 8: Decent Work and Economic Growth	Transparent, constraint-aware tools support informed saving decisions; held-out testing discourages over-confident claims.
SDG 9: Industry, Innovation and Infrastructure	Bio-inspired computing is applied to a financial decision problem and delivered as tested, reusable software.

1.6 Existing System

Quadratic programming solvers give one efficient portfolio per risk aversion but do not handle cardinality directly. Single-objective heuristics, including PSO [9], [10], [11] and genetic algorithms [3], handle realistic constraints but optimise one scalarised objective, so each trade-off needs a new run. Multi-objective evolutionary algorithms return a front in one

run [12], [13] (reviewed in [14]). These systems usually lack a common evaluation protocol, mostly evaluate in-sample, and rarely come with software a student can run and inspect.

1.7 Proposed System

SwarmFolio has four parts: (i) a portfolio model with three objectives and a repair operator that turns any real vector into feasible weights; (ii) MOPSO and NSGA-II, which share that model and therefore search the same feasible space; (iii) an evaluation layer with baselines, front-quality indicators, selection rules and held-out back-testing; and (iv) a web workbench that animates the optimisers while they search. What distinguishes it from the existing systems is the shared repair operator, the three-objective formulation with CVaR, a hypervolume normalised across all methods in a run, and a held-out study over 30 markets.

1.8 Work Plan

The work was organised into nine overlapping activities over a fifteen-week semester, as shown below.

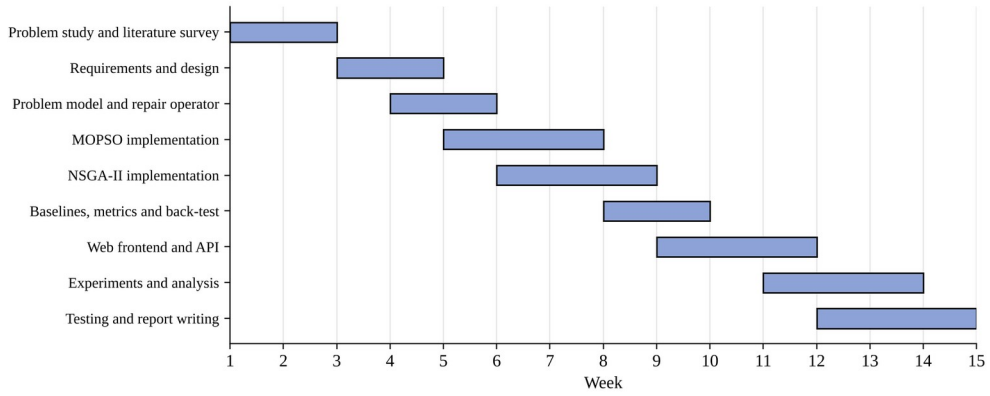


Figure 1.1: Work plan of the project (weeks of the semester).

Chapter 2

Literature Survey

2.1 Overview

The relevant literature falls into four groups: the financial foundations, constrained and multi-objective formulations, the metaheuristics used to solve them, and the assessment of Pareto fronts. Each group is summarised below in the authors' own words, followed by the research gap.

2.2 Detailed Survey

2.2.1 Foundations of portfolio selection

Markowitz [1] introduced mean-variance selection and the efficient frontier. Sharpe [15] proposed the reward-to-variability ratio used here to pick one portfolio from a front. Rockafellar and Uryasev [2] introduced CVaR optimisation, a tail-risk measure used here as a third objective. Michaud [6] argued that optimisers behave as error maximisers, and DeMiguel et al. [7] reported that optimised strategies did not consistently beat equal weighting out of sample; both motivate the held-out evaluation.

2.2.2 Constrained and multi-objective formulations

Chang et al. [3] studied the cardinality-constrained frontier with genetic algorithms, tabu search and simulated annealing and showed that such constraints make the frontier discontinuous and hard to compute exactly. Anagnostopoulos and Mamanis [12] treated selection as a tri-objective problem over risk, return and the number of securities, and Skolpadungket et al. [13] applied multi-objective genetic algorithms with realistic constraints. Metaxiotis and Liagkouras [14] review the field.

2.2.3 Evolutionary and swarm algorithms

Holland [16] established the genetic algorithm. Deb et al. [5] proposed NSGA-II (fast non-dominated sorting, crowding distance, elitist survival), and Deb and Agrawal [17] the simulated binary crossover. Kennedy and Eberhart [4] introduced PSO, Shi and Eberhart [18] the inertia weight, and Coello Coello et al. [19] a multi-objective PSO with an external repository. For portfolios, Cura [9], Zhu et al. [10] and Deng et al. [11] applied PSO to the constrained problem with a single scalarised objective.

2.2.4 Assessing the quality of a Pareto front

Zitzler et al. [20] analysed performance indicators for multi-objective optimisers; the hypervolume, which originates in [21], is compatible with Pareto dominance and is the

primary indicator here. Schott [22] proposed the spacing metric for how evenly a front is spread. Deb [23] is the standard reference.

Table 2.1: Summary of the literature used in the project

Ref.	Focus	Method	Contribution used here
[1]	Portfolio theory	Mean-variance	Return-risk trade-off
[2]	Tail risk	CVaR optimisation	Third objective
[3]	Cardinality	GA, tabu search, annealing	Motivation for heuristics
[12], [13]	Multi-objective portfolios	MOEAs	Multi-objective formulation
[5]	Algorithm	NSGA-II	Basis of the GA
[19]	Algorithm	Multi-objective PSO	Basis of the swarm optimiser
[9], [10], [11]	Portfolio PSO	Single-objective PSO	PSO suits the constrained problem
[6], [7]	Out-of-sample evidence	Empirical comparison	Held-out tests, 1/N baseline
[20], [22]	Indicators	Hypervolume, spacing	Evaluation of fronts

2.3 Research Gap Identification

- **Evaluation protocol.** The PSO and GA portfolio studies surveyed report benchmark data and mainly in-sample results; held-out results over many markets with a paired test against equal weight were not found.
- **Confounded comparison.** Constraint handling differs between algorithms, so a difference in outcome cannot be attributed to the search strategy alone.
- **Tail risk with swarms.** The surveyed portfolio PSO studies [9], [10], [11] optimise one scalarised objective, so a three-objective front with CVaR is not produced.
- **Inspectability.** Few surveyed works ship software in which the search can be observed and a portfolio inspected.

These gaps describe the papers surveyed here, which are not an exhaustive review of the field.

2.4 Mapping of your Objective with Research Gap Identification

Table 2.2: Mapping of objectives to research gaps

Objective (Section 1.3)	Gap addressed	How the project addresses it
1. Three-objective model with constraints	Tail risk	Return, volatility and CVaR optimised together under cap, floor and cardinality constraints.
2. Shared repair operator, MOPSO and NSGA-II	Confounded comparison	Both algorithms call the same repair operator and objective function.
3. Front-quality indicators and baselines	Evaluation protocol	Shared-normalisation hypervolume, spacing, runtime, random search at equal budget.
4. Constraint, scale, budget and ablation studies	Evaluation protocol	Experiments E2, E3, E4 and E6 (Section 4.2.1).
5. Held-out test over many markets	Evaluation protocol	30 simulated markets, paired Wilcoxon test against equal weight (Chapter 7).
6. Browser workbench	Inspectability	Live animation of both populations and a click-to-inspect panel.

Chapter 3

Requirement Analysis and Specification

3.1 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement
FR1	Accept a market from the synthetic generator (assets, days, seed) or from an uploaded CSV of prices.
FR2	Let the user set the maximum weight, minimum holding size, optional holding limit, population size, generations and training share; reject impossible combinations.
FR3	Run MOPSO and NSGA-II on the same problem and return feasible, non-dominated portfolios (long-only, fully invested, within cap and limit).
FR4	Show the progress of both searches live, including the current population and front, and allow the user to stop a job.
FR5	Compute hypervolume, spacing, front size, best Sharpe ratio and runtime, with random search as a baseline.
FR6	Let the user select a portfolio by click, slider or rule (best Sharpe, lowest volatility, balanced) and show its weights.
FR7	Show training and held-out return, volatility, Sharpe ratio and drawdown next to equal weight, with a held-out wealth curve.
FR8	Report invalid input and server errors in the interface in plain language.
FR9	Provide the same functions from Python and from a command-line tool that writes tables and figures.

3.2 Non-functional Requirements

Table 3.2: Non-functional requirements

Category	Requirement
Correctness	Constraints hold for every portfolio produced; objective values match an independent calculation (covered by tests).
Reproducibility	The same data, parameters and seed give the same front.
Performance	A default run (100 individuals, 150 iterations, 20 assets, 1,500 days) finishes in about ten seconds or less.
Usability	Plain-language labels, keyboard operation, phone-to-desktop layout, light and dark themes.
Portability	Standard-library server and a build-free interface that needs no network access.
Security	Inputs validated and bounded; static files only from the web directory; user text escaped (Section 3.7.1).

3.3 Hardware Requirements

The project is software only. Development and experiments ran in a cloud container with four virtual CPU cores. A dual-core 64-bit laptop with 4 GB RAM and 500 MB of free disk is enough to use it, and no network is needed to run it.

3.4 Software Requirements

Table 3.3: Software requirements

Software	Version used	Purpose
Python	3.13	Library, server and experiments
NumPy [24]	2.5	Vectorised evaluation and the optimisers
pandas [25]	3.0	Time-indexed returns, CSV import
Matplotlib [26]	3.11	Plots written by the command-line tool and the report figures
SciPy [27]	1.18	Wilcoxon test in the report analysis only
pytest	-	Automated tests
Web browser	Any current version	HTML, CSS and JavaScript interface (canvas)

3.5 Cost Estimation

All software is free and open source, so the incremental cost is small. Printing and binding of the report depends on the department’s requirements.

Table 3.4: Cost estimates

Item	Estimated cost (INR)	Note
Python and open-source libraries	0	Open-source licences
Web interface (HTML, CSS, JavaScript)	0	No framework or build tool
Cloud container for development	0	Provided with the environment used
Printing and binding of the report	to be filled by the team	Department requirements

3.6 Assumptions

- Daily returns are the unit of analysis (252 trading days a year); the training-window empirical distribution is an adequate basis for estimating return, volatility and CVaR. The held-out tests probe this assumption.
- Positions are long-only and fully invested, with no leverage or short selling.

- Trading costs are ignored in the optimisation; a flat cost on turnover is used only in the optional walk-forward test.
- The simulated markets are a controlled testbed and not a model of any particular exchange.

3.7 Risks and Challenges

Table 3.5: Risks and mitigations

Risk	Effect	Mitigation
Over-fitting to the training window	Strong in-sample, weaker on unseen data	Held-out evaluation, equal-weight baseline, 30 markets
Constraint infeasibility	Impossible setting, e.g. 10% cap with five holdings	Validation in the interface and the problem constructor
Premature convergence	Population collapses onto few holdings	Turbulence and swap mutation, tested by ablation (Section 7.4)
Metric misuse	Hypervolume depends on the reference point	One shared normalisation per comparison
Synthetic data not representative	Conclusions may not transfer	Stated as a limitation; CSV import allows real data

3.7.1 AI Security Challenges

Although there is no learned model, the system accepts user data and returns decision support, so several concerns apply. Their handling is described in Section 5.4.

- **Malformed or oversized input:** an uploaded CSV could be malformed or crafted to exhaust memory or time.
- **Injection into the interface:** column names from a file are shown as asset names and could carry script.
- **Resource exhaustion and path traversal:** unbounded parameters or jobs, or a file handler tricked into serving files outside the web directory.
- **Data manipulation and access:** outlier prices can bias the optimiser, and the server has no authentication and is meant for local use.

Chapter 4

Design

4.1 High Level Design

4.1.1 System Architecture

SwarmFolio has three layers (figure below). The browser presents controls and charts and exchanges JSON with a small server. The server owns the jobs: it runs each optimisation on worker threads, keeps the latest snapshot of each population and returns it when the browser polls. The optimisation core is an ordinary Python package with no dependence on the server, so the command-line tool, the tests and the web application all use the same code.

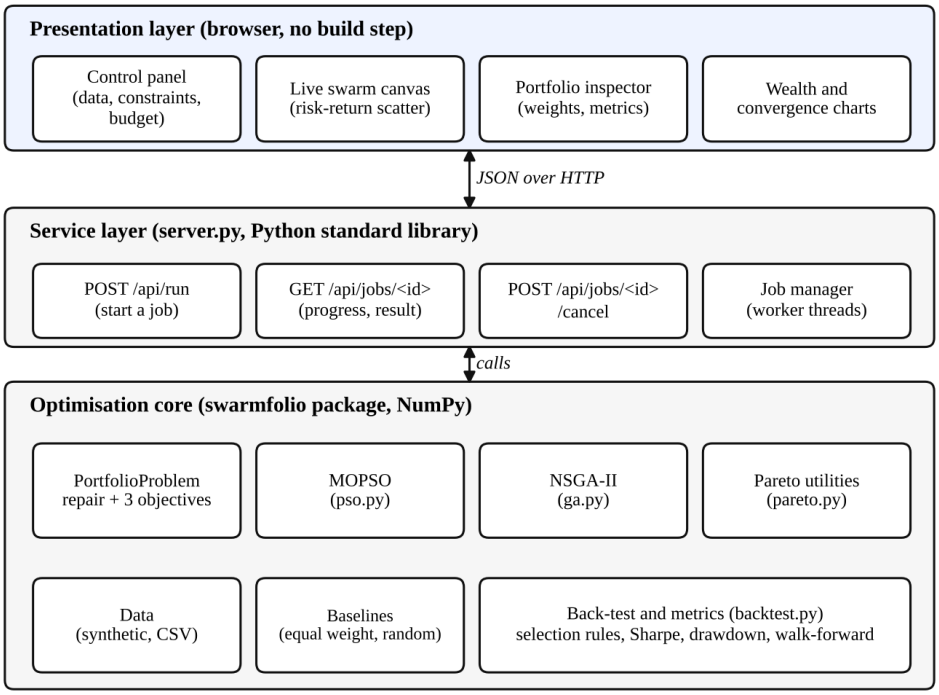


Figure 4.1: Three-layer architecture of SwarmFolio.

A run proceeds as follows. The browser sends the configuration; the server validates it, builds the market and the problem, and starts MOPSO and NSGA-II on separate threads. Each optimiser calls a callback every iteration that records its population and front. The browser polls about nine times a second and redraws. When both finish, the server runs the random-search baseline, scores every front on the held-out data and publishes the result.

4.1.2 Abstract specification of Sub-systems

Table 4.1: Sub-systems and their responsibilities

Sub-system (module)	Responsibility	Inputs and outputs
Data (data.py)	Synthetic market or CSV loader	Seed and size, or a file; daily returns
Problem (problem.py)	Objectives, constraints, repair, evaluation	Returns and settings; weights in, objectives out
MOPSO (pso.py)	Multi-objective swarm search with archive	Problem and parameters; non-dominated set
NSGA-II (ga.py)	Multi-objective genetic search	Problem and parameters; non-dominated set
Pareto, baselines, back-test	Sorting, hypervolume, spacing; equal weight and random search; selection and performance metrics	Objective matrices, fronts and held-out returns
Server (server.py)	Job manager, JSON API, static files	HTTP requests; JSON responses
Web interface (web/)	Controls, live canvas, inspector, charts	User actions; API responses

4.1.3 Interface Design

The interface is a single page with a control rail on the left and a working area on the right; on narrow screens the working area comes first. Colour is used for data only (blue for the swarm, orange for the genetic algorithm, grey for random portfolios), and light and dark themes follow the system setting. The stage plots annual return against volatility (or CVaR) and shows both populations while they search; afterwards it shows the two fronts, an equal-weight marker and a ring on the selected portfolio. Below it, the inspector shows weights, training against held-out performance and a held-out wealth curve. If a setting is impossible, for example a 10% cap with five holdings, a message appears under the field and the run button is disabled.

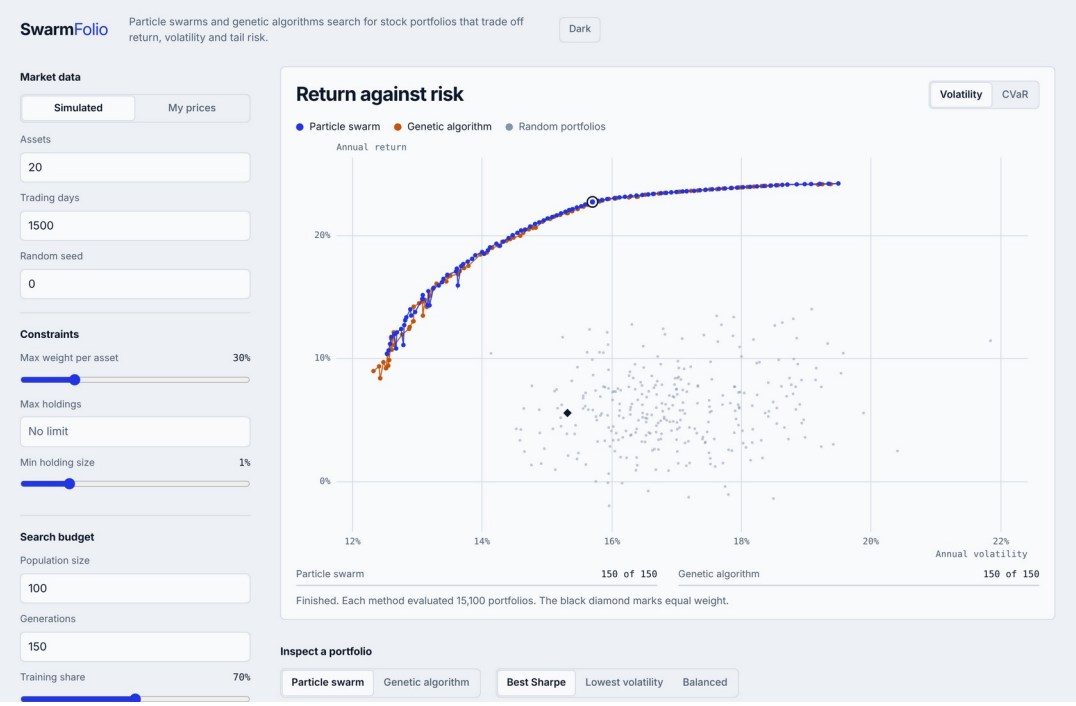


Figure 4.2: Main screen after a run: control rail on the left, risk-return stage on the right.



Figure 4.3: Portfolio inspector with selection controls, performance table and weights.

Table 4.2: HTTP interface of the server

Endpoint	Method	Purpose
/api/run	POST	Start a job from a JSON configuration; returns the job id or an error message.
/api/jobs/<id>	GET	Poll status, progress, the latest population and front of each algorithm and, when done, the full result.
/api/jobs/<id>/cancel	POST	Stop a running job.

4.2 Methodology

4.2.1 Research Design

The study is a controlled computational experiment. All algorithms see the same training data and constraints and get the same number of objective evaluations. Six experiments were run with 3 to 30 independent seeds.

Table 4.3: Experimental design

ID	Question	Design
E1	Do both algorithms beat random search, and do they differ?	Default setting (20 assets, 1,500 days), 10 seeds
E2	How do tighter holding limits affect front quality?	K of none, 10, 8 and 5 holdings, 5 seeds each
E3	How do results and run time scale with assets?	10, 20, 40 and 60 assets, 3 seeds each
E4	How fast does front quality improve with budget?	5 to 200 iterations, 5 seeds each
E5	Do chosen portfolios beat equal weight on unseen data?	30 simulated markets; best-Sharpe and minimum-volatility picks
E6	Do the turbulence and swap-mutation operators help?	Full against operator removed, K = 8, 8 seeds

Indicators. Hypervolume is the fraction of a normalised objective box dominated by a front. The box runs from the best value of each objective over all fronts compared to a reference point just beyond the worst, so methods compared in one run share a scale; it is estimated with 200,000 Monte Carlo samples. Spacing is the Schott metric [22], front size is the number of non-dominated points, and runtime is wall-clock time. **Held-out metrics:** annualised return is 252 times the mean daily return, volatility is the daily standard deviation times the square root of 252, the Sharpe ratio is their quotient (risk-free rate zero) and maximum drawdown is the largest peak-to-trough fall of wealth. **Statistics:** paired Wilcoxon signed-rank tests [28] from SciPy [27].

4.2.2 Data Collection

No licensed market data was used. The experiments use a seeded synthetic market, which makes every result reproducible and allows many independent markets. Returns of asset i on day t follow a market-plus-sector factor model with fat-tailed noise:

$$r_{i,t} = \alpha_i + \beta_i\, m_t + s_{k(i),t} + \sigma_i\, \varepsilon_{i,t} \qquad (1)$$

Here m_t is the market factor, $s_{k(i),t}$ the asset's sector factor and ε a standardised Student-t variable with five degrees of freedom. A crisis regime between 45% and 51% of the sample

multiplies returns by 2.2 and adds negative drift. The CSV loader accepts a wide file of adjusted close prices, drops tickers with more than 5% missing values, forward-fills gaps and converts to daily returns; the same 70/30 split applies.

Table 4.4: Parameters of the synthetic market

Component	Setting
Market factor m_t	Mean 7% a year; daily volatility 0.9%; Student-t, 5 d.o.f.
Sector factors	4 sectors; daily volatility 0.5%
Beta β_i and alpha α_i	Uniform on [0.5, 1.4]; uniform on [-2%, 10%] a year
Idiosyncratic volatility σ_i	Uniform on [0.6%, 2.0%] a day
Crisis regime	Days 45% to 51% of the sample: returns x 2.2, drift -0.15% x beta a day
Default size and split	20 assets, 1,500 business days; first 1,050 days train, last 450 days test

Chapter 5

Implementation

5.1 Pseudo code

This section first explains, in plain words, what the two optimisation algorithms are, what they do in this project and how they work, and then gives the pseudo code of the implementation.

5.1.1 Particle Swarm Optimization (PSO)

What it is. Particle Swarm Optimization is a search method inspired by a flock of birds looking for food. No bird knows where the food is, but each one remembers the best place it has found and watches where its neighbours have done well, and the flock gradually drifts toward good areas [4]. In SwarmFolio each bird, called a particle, is one candidate portfolio: a list of weights, one per stock, that add up to 100%. A swarm of 100 particles moves around the space of possible portfolios.

What it does here. The swarm looks for portfolios with high return and low volatility and CVaR. Because these goals conflict, it does not seek one best portfolio. It builds a set of non-dominated portfolios (the Pareto front), where no portfolio can be improved on one goal without getting worse on another. This multi-objective version keeps an external archive of such portfolios and lets it guide the swarm [19].

How it works. Each particle has a position (its current weights) and a velocity (the direction and size of its next step). Every iteration, for each particle:

1. **Score** the portfolio on the three objectives and remember the particle's own best portfolio so far (its personal best).
2. **Pick a leader** from the archive of non-dominated portfolios. Leaders in sparsely populated parts of the front are preferred, which keeps the front spread out.
3. **Move.** The new velocity combines three pulls: momentum (keep going the same way), the particle's own best, and the leader. The particle then steps along that velocity (equations 2 and 3).
4. **Repair and shake.** The new weights are repaired so they stay legal (long-only, capped, within the holding limit), and with small probability random noise (turbulence) is added so that the swarm does not pile onto the same few stocks.
5. **Update the archive**: add new non-dominated portfolios, remove dominated and duplicate ones, and trim the most crowded points.

$$v \leftarrow w \cdot v + c_1 r_1 (pbest - x) + c_2 r_2 (leader - x) \quad (2)$$

$$x \leftarrow x + v \quad (3)$$

Here x is the particle's weights, v its velocity, r_1 and r_2 random numbers between 0 and 1, and $c_1 = c_2 = 1.5$. The inertia weight w falls from 0.9 to 0.4 [18], so the swarm explores widely at first and refines later. After 150 iterations the archive is the result.

5.1.2 Genetic Algorithm (NSGA-II)

What it is. A genetic algorithm copies natural selection [16]. Better candidates are more likely to become parents, parents mix their genes to make children, random mutation adds novelty, and over many generations the population improves. NSGA-II [5] is the standard version for several objectives. In SwarmFolio each individual is a portfolio, and its genes are the weights of the stocks.

What it does here. Like the swarm, it searches for a front of non-dominated portfolios. Since "better" cannot be a single number with three objectives, NSGA-II ranks individuals by dominance. Rank 0 holds the portfolios that no other portfolio dominates; remove them and the next set is rank 1, and so on. Among equals, the one in a less crowded region wins, using the crowding distance, so that the front stays diverse.

How it works. A population of 100 portfolios goes through the following steps each generation:

1. **Rank** every individual by non-domination and compute its crowding distance.
2. **Select parents** by tournament: pick two at random and keep the one with the better rank, or the larger crowding distance if the ranks are equal.
3. **Crossover.** Simulated binary crossover [17] blends the weights of two parents into two children that lie near the parents.
4. **Mutate.** Small random changes are made to the weights, and a swap mutation moves the whole weight of one holding to another stock, which changes which stocks are held.
5. **Repair** the children so they are legal portfolios, and score them.
6. **Survive.** Parents and children (200 in all) are pooled and the best 100 by rank, then crowding distance, are kept. Because good portfolios are never discarded (elitism), the front can only improve.

After 150 generations, the rank-0 individuals are the result.

5.1.3 How the two algorithms differ

Table 5.1: Comparison of the two algorithms

Aspect	MOPSO (particle swarm)	NSGA-II (genetic algorithm)
Inspired by	Flocking of birds or fish	Natural selection and genetics
New candidates	Particles move: velocity pulled by own best and an archive leader	Selected parents are crossed and mutated into children
What guides the search	Leaders drawn from the archive of non-dominated portfolios	Rank by dominance, then crowding distance
Memory	Personal best of each particle plus the archive	The whole population, with elitist survival
Keeps the front spread out	Crowding-based leader choice and archive pruning	Crowding distance in selection and survival
Strength in our results	Converges fast; best at small budgets	Slightly better from about 100 iterations

Table 5.2: Parameter settings used for both algorithms

Setting	MOPSO	NSGA-II
Population and iterations	100 particles, 150 iterations	100 individuals, 150 generations
Main parameters	Inertia 0.9 to 0.4, $c_1 = c_2 = 1.5$, maximum velocity 0.3, turbulence 10%	Crossover probability 0.9, SBX index 15, mutation index 20, swap mutation 0.2
Result kept	Archive of up to 100 portfolios	Rank-0 front of the final population
Evaluations	15,100 per run	15,100 per run

5.1.4 Pseudo code of the implementation

For a portfolio w in R^N and a matrix R of T daily returns, the three objectives that are minimised are

$$f_1(w) = -252 \cdot \text{mean}(Rw) \tag{4}$$

$$f_2(w) = \sqrt{252} \cdot \text{std}(Rw) \tag{5}$$

$$f_3(w) = -(\text{mean of the k lowest values of } Rw) \tag{6}$$

where k is the ceiling of 0.05 T. All three are computed for a whole population at once as a matrix product followed by a sort. Algorithm 1 is the repair operator shared by both optimisers; Algorithms 2 and 3 are the optimisers themselves.

Algorithm 1: Constraint repair

```
function REPAIR(x):                      # any real vector -> feasible weights
    w <- max(x, 0)
    if K is set: keep the K largest entries of w, zero the rest
    normalise w to sum 1; set entries below w_min to 0
    if fewer than ceil(1/w_max) entries are non-zero:
        activate further assets in order of x with a small weight
    normalise w to sum 1
    repeat while some w_i > w_max:
        excess <- sum over i of (w_i - w_max); set those w_i to w_max
        give the excess to the other non-zero assets in proportion to w
        (if all are capped, give it to the next assets in order of x)
    return w / sum(w)
```

Algorithm 2: Multi-objective particle swarm optimiser

```
function MOPSO(problem, N, T, archive_size):
    X <- REPAIR(random vectors); V <- small random velocities
    F <- EVALUATE(X); PBEST <- X; ARCHIVE <- non-dominated(X, F)
    for t = 1..T:
        w <- 0.9 - 0.5 * t/T                      # linearly decreasing inertia
        for each particle i:
            leader_i <- binary tournament on ARCHIVE by crowding distance
            V_i <- w*V_i + 1.5*r1*(PBEST_i - X_i) + 1.5*r2*(leader_i - X_i)
            clip V_i to [-0.3, 0.3]; X_i <- X_i + V_i
            with probability 0.1: add Gaussian noise (sd 0.1) to 30% of entries
        X <- REPAIR(X); F <- EVALUATE(X)
        for each i: if F_i dominates PBEST_i, or neither dominates (50% chance):
            PBEST_i <- X_i
        ARCHIVE <- non-dominated(ARCHIVE + X); remove duplicates
        while |ARCHIVE| > archive_size: remove the most crowded point
    return ARCHIVE
```

Algorithm 3: NSGA-II genetic algorithm

```
function NSGA2(problem, P, G):
    X <- REPAIR(random vectors); F <- EVALUATE(X)
    for g = 1..G:
        rank, crowd <- NON_DOMINATED_SORT(F), CROWDING_DISTANCE(F)
        parents <- binary tournament (lower rank wins; ties: larger crowding)
        children <- SBX crossover(parents, eta_c=15, p_c=0.9)
        children <- polynomial mutation(eta_m=20); swap mutation with p=0.2
        children <- REPAIR(children); F_c <- EVALUATE(children)
        pool <- X + children; sort pool by (rank, -crowding)
        X, F <- first P members of the sorted pool
    return rank-0 members of X
```

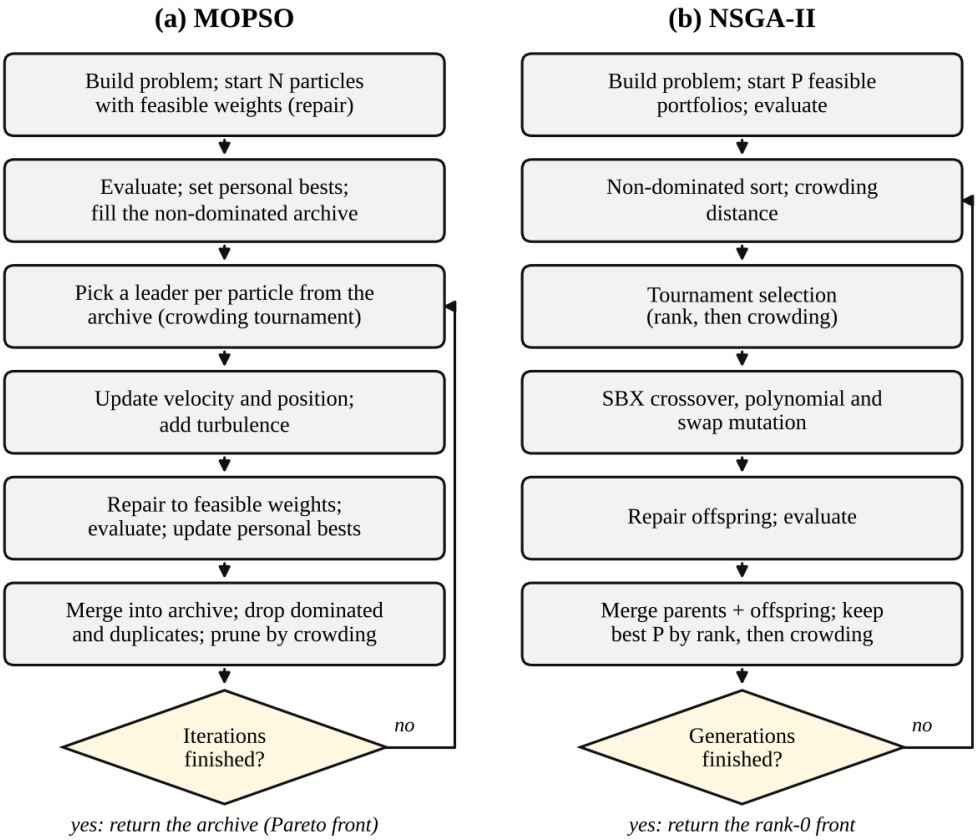


Figure 5.1: Flowcharts of (a) the multi-objective particle swarm optimiser and (b) the NSGA-II genetic algorithm.

5.2 Use of appropriate advanced tools and techniques

Table 5.3: Techniques and tools

Technique or tool	Where it is used	Reason
Fast non-dominated sorting, crowding distance [5]	NSGA-II; MOPSO leader choice and archive pruning	Pareto ranking in one pass; keeps the front spread out
External archive [19]	MOPSO	Stores the non-dominated solutions that guide the swarm
Inertia weight schedule [18]	MOPSO	Explores early and refines late
Simulated binary crossover [17]	NSGA-II	Real-coded crossover with controlled spread
Vectorised evaluation (NumPy [24])	All objective calculations	One matrix product scores a whole population
Monte Carlo hypervolume, seeded generators	Front comparison and every stochastic step	Three-objective hypervolume without extra libraries; exact reproducibility
Worker threads and a canvas interface	Server and browser	Both searches run side by side, can be cancelled and are animated smoothly

5.3 Novelty

Both optimisers are established methods, so the contribution lies in how they are set up, compared and presented:

- **One feasible space for both optimisers:** a single repair operator is applied to every position, offspring and initial candidate.
- **Three-objective formulation with CVaR** solved by a swarm and a GA in one framework, with cap, floor and cardinality constraints.
- **Shared-normalisation hypervolume, operator ablations and a 30-market held-out study** with paired tests.
- **Observable search:** the interface animates both populations and lets the user inspect any portfolio on the fronts.

5.4 Mitigation of AI Security threats

Table 5.4: Security threats and mitigations

Threat (Section 3.7.1)	Mitigation implemented	Verified by
Malformed or oversized input	Bodies limited to 8 MB; JSON object required; numeric parameters range-checked; CSV errors returned as messages	Server tests; upload test
Script injection through asset names	All text from uploaded files is escaped before it enters the page	Code review
Resource exhaustion	Parameter bounds; a new run cancels earlier jobs; only the eight latest jobs are kept; cancel endpoint	Stop-button test
Path traversal	Requested paths must resolve inside the web directory, otherwise 404	Server test
Data manipulation; unauthorised access	Held-out evaluation and equal-weight baseline expose over-fitting; cap and limit reduce concentration; local-only default, no accounts	Experiment E5; default bind address

These measures suit a locally run teaching and research tool. A hosted version would need authentication, rate limiting and an isolated worker process.

Chapter 6

Testing and Validation

6.1 Data interpretation

The default market has 20 assets and 1,500 business days. The first 1,050 days form the training window on which every optimiser works, and the last 450 days are held out and used only to evaluate chosen portfolios. On the training window the single assets have annualised returns from -13.2% to 35.9% and volatilities from 16.5% to 41.2%, so a diversified portfolio can reach a much lower volatility than any single asset.

Return is the mean daily return times 252, volatility is the daily standard deviation times the square root of 252, and CVaR is the average loss on the worst 5% of days (a CVaR of 1.9% means such days lost 1.9% on average). A point on the front is better than another if it is no worse on all three objectives and better on at least one. Two cautions apply: training numbers are in-sample and only held-out numbers speak about unseen data, and hypervolumes are comparable only between methods sharing one normalisation, not between scenarios.

6.2 Analysis

Verification was organised in three levels.

- **Component level:** the repair operator is tested on random, all-zero, all-one and unit vectors with and without a cardinality limit; the objective function is checked against an independent calculation; the Pareto utilities against a worked example.
- **Algorithm level:** each optimiser must return only feasible, mutually non-dominated portfolios whose stored objectives match a re-evaluation, must be reproducible for a fixed seed, and must beat random search at equal budget.
- **System level:** the server is tested through a complete job, error paths and the HTTP interface, and the interface is exercised in a real browser.

Across all fronts of experiment E2, which includes limits of 10, 8 and 5 holdings, there were 0 violations of the 30% cap or the holding limit.

6.3 Test results

6.3.1 Testing process

The automated tests are written with pytest, run in a few seconds and are executed after each change; all 14 passed in the final run for this report. Interface tests ran in headless Chromium

(Playwright) against the running server in light desktop, dark desktop and phone-sized windows.

6.3.2 System test cases

Table 6.1: System test cases and results

ID	Test case	Expected result	Result
T1	Repair on random, zero, ones and unit vectors; no limit and K = 5	Non-negative weights summing to 1, each at most 30%, at most K non-zero	Pass
T2	A 10% cap with a limit of 5 holdings	Error that the configuration is infeasible	Pass
T3	Objectives of equal weight against a manual calculation	Return, volatility and CVaR agree	Pass
T4	Dominance mask, ranks, crowding distance and hypervolume on a worked example	Expected values	Pass
T5	MOPSO and NSGA-II with a limit of 6 holdings	Feasible, non-dominated, objectives match a re-evaluation	Pass (both)
T6	Both algorithms against random search at equal budget	Larger hypervolume for both	Pass
T7	Two MOPSO runs with the same seed; a command-line run	Identical fronts; tables and figures written	Pass
T8	Complete job; infeasible job; 500 assets; HTTP requests including a parent-directory path	Full result; clear errors; traversal path returns 404	Pass
T9	Browser: load, hover and click a point, switch algorithm, rule and axis	Fronts drawn; tooltip, weights and metrics update	Pass
T10	Browser: 10% cap with 5 holdings; CSV without and with a file; Stop	Inline message and disabled Run; CSV run completes; run stops	Pass
T11	Browser: phone width 390 px and dark scheme	No horizontal scrolling (found and fixed); text readable	Pass

Chapter 7

Results and Discussion

7.1 Presentation of Results

This section presents the six experiments of Section 4.2.1. Values after the plus-minus sign are sample standard deviations over seeds, and all algorithms received the same data, constraints and evaluation budget.

E1: Front quality at the default setting

Table 7.1: Front quality over 10 seeds (20 assets, 100 particles or individuals, 150 iterations)

Method	Hypervolume	Front size	Spacing	Best Sharpe	Seconds	Evaluations
Particle swarm (MOPSO)	0.702 ± 0.027	100.0	0.0103	1.448	1.5	15,100
Genetic algorithm (NSGA-II)	0.706 ± 0.030	98.0	0.0160	1.447	1.3	15,100
Random search	0.254 ± 0.058	20.1	0.1271	0.987	<0.1	15,100

Both algorithms dominate random search (hypervolume about 0.70 against 0.25, Wilcoxon $p = 0.002$); random search finds only about 20 non-dominated points against about 100. NSGA-II is marginally higher than MOPSO (difference 0.0042, higher in 9 of 10 seeds, $p = 0.014$), which is small next to the seed-to-seed spread of about 0.027. MOPSO gives a more evenly spread front (spacing 0.0103 against 0.0160, $p = 0.002$). Run times, measured separately with nothing else running, are about 1.5 s for MOPSO and 1.3 s for NSGA-II.

Selected portfolios on the default market

Table 7.2: Selected portfolios on the default market (seed 0)

Portfolio	Held	Train return	Train vol.	Train Sharpe	Held-out CAGR	Held-out vol.	Held-out Sharpe	Held-out max DD
MOPSO best sharpe	4	22.7%	15.7%	1.448	-5.5%	13.6%	-0.350	-24.6%
MOPSO lowest volatility	12	10.4%	12.5%	0.828	-3.5%	11.2%	-0.261	-15.3%
MOPSO balanced	7	19.0%	14.1%	1.348	-4.9%	12.3%	-0.346	-19.7%
NSGA-II best sharpe	4	22.8%	15.7%	1.448	-5.6%	13.6%	-0.354	-24.6%
NSGA-II lowest volatility	12	9.0%	12.3%	0.727	-5.0%	11.1%	-0.406	-17.3%
NSGA-II balanced	6	19.0%	14.2%	1.342	-5.5%	12.3%	-0.397	-20.2%
Equal weight	20	5.6%	15.3%	0.362	-7.6%	13.5%	-0.521	-21.5%

All strategies lost money on this held-out window, including equal weight, because it is a weak stretch of the simulated path; the optimised portfolios lost less (for example -5.5% a year against -7.6%), but one market cannot separate skill from luck, hence experiment E5. The best-Sharpe portfolios hold four assets at the 30% cap and the lowest-volatility ones hold twelve.

E2: Effect of the holding limit

Table 7.3: Hypervolume against the holding limit (5 seeds each; 20 assets; scale shared within each row)

Max holdings K	MOPSO	NSGA-II	Random	Best Sharpe	Most holdings seen
5	0.749 ± 0.026	0.742 ± 0.029	0.433 ± 0.045	1.448	5 / 5
8	0.731 ± 0.031	0.732 ± 0.032	0.330 ± 0.031	1.448	8 / 8
10	0.715 ± 0.028	0.716 ± 0.032	0.299 ± 0.052	1.448	10 / 10
None	0.700 ± 0.030	0.703 ± 0.034	0.263 ± 0.058	1.448	12 / 14

The limit was respected in every run and the cap was never exceeded (0 violations). The best Sharpe ratio is the same for every K because the best portfolio needs only four assets, so even K = 5 does not bind. Random search improves as K falls because the feasible region shrinks, whereas the two optimisers stay close to each other. Hypervolumes must not be compared across K.

E3: Number of assets

Table 7.4: Hypervolume and run time against the number of assets (3 seeds; run time measured separately)

Assets	MOPSO	NSGA-II	Random	MOPSO seconds	NSGA-II seconds
10	0.677 ± 0.019	0.682 ± 0.019	0.546 ± 0.068	1.51	1.31
20	0.581 ± 0.004	0.581 ± 0.006	0.424 ± 0.057	1.49	1.27
40	0.672 ± 0.027	0.668 ± 0.025	0.232 ± 0.082	1.38	1.33
60	0.617 ± 0.011	0.590 ± 0.004	0.124 ± 0.041	1.40	1.24

Run time stays between about 1.2 and 1.5 seconds from 10 to 60 assets because evaluation is one matrix product. The advantage over random search grows with dimension: at 60 assets the optimisers reach about 0.62 and 0.59 against 0.12, whereas at 10 assets the gap is much smaller.

E4: Search budget

Table 7.5: Hypervolume against budget (5 seeds each; one fixed reference for all rows)

Iterations	Evaluations	MOPSO	NSGA-II	Random search
5	600	0.628 ± 0.040	0.369 ± 0.036	0.284 ± 0.051
10	1,100	0.752 ± 0.010	0.463 ± 0.049	0.330 ± 0.068
25	2,600	0.788 ± 0.003	0.646 ± 0.033	0.353 ± 0.075
50	5,100	0.797 ± 0.005	0.769 ± 0.010	0.381 ± 0.075
100	10,100	0.801 ± 0.002	0.804 ± 0.001	0.395 ± 0.076
200	20,100	0.802 ± 0.002	0.811 ± 0.002	0.430 ± 0.096

MOPSO is much better at small budgets: with 10 iterations (1,100 evaluations) it reaches 0.75, which NSGA-II needs 25 to 50 iterations to match, and it was ahead in all 5 seeds at 50 iterations or fewer. From 100 iterations on, NSGA-II was ahead in all 5 seeds (0.811 against 0.802 at 200). With five seeds the smallest attainable p value is 0.062, which is reported for these unanimous comparisons.

E5: Held-out performance over 30 markets

Table 7.6: Held-out performance over 30 independent markets (mean over markets; p is the paired Wilcoxon test of held-out Sharpe against equal weight)

Strategy	Train Sharpe	Held-out Sharpe	Median	Held-out volatility	Held-out max DD	Wins vs EW	p vs EW
Equal weight	0.52	0.59 ± 0.76	0.50	14.7%	-16.2%	-	-
MOPSO best Sharpe	1.34	0.53 ± 0.81	0.56	16.6%	-19.0%	14/30	0.428
NSGA-II best Sharpe	1.34	0.53 ± 0.79	0.56	16.7%	-19.0%	13/30	0.416
Random search best Sharpe	1.06	0.51 ± 0.76	0.50	15.8%	-18.9%	11/30	0.221
MOPSO min volatility	0.64	0.61 ± 1.01	0.59	12.6%	-15.1%	12/30	0.871
NSGA-II min volatility	0.57	0.62 ± 1.00	0.59	12.4%	-14.6%	16/30	0.626
Random search min volatility	0.56	0.66 ± 0.89	0.58	13.2%	-14.9%	18/30	0.164

Best-Sharpe portfolios. The mean training Sharpe ratio is 1.34 for MOPSO and 1.34 for NSGA-II, but on held-out data it falls to 0.53 and 0.53, no better than equal weight (0.59). The differences from equal weight are not significant, MOPSO and NSGA-II do not differ ($p = 0.641$), and random search with a much poorer front does about as well.

Lowest-volatility portfolios. Held-out volatility is lower than that of equal weight in 30 of 30 markets for MOPSO and 30 of 30 for NSGA-II (means 12.6% and 12.4% against 14.7%, $p < 0.001$). Maximum drawdown is shallower in 19 markets for MOPSO ($p = 0.061$) and 24 for NSGA-II ($p = 0.005$), with a Sharpe ratio similar to equal weight.

E6: Operator ablation

Table 7.7: Effect of removing an operator (8 seeds; limit of 8 holdings)

Variant	Hypervolume
PSO (full)	0.6607 ± 0.0131
PSO without turbulence	0.6589 ± 0.0131
GA (full)	0.6546 ± 0.0139
GA without swap mutation	0.6530 ± 0.0093

Removing the PSO turbulence step changed the hypervolume by 0.0017 on average ($p = 0.641$) and removing the GA swap mutation by 0.0016 ($p = 0.547$); neither is distinguishable from seed-to-seed variation at this setting.

7.1.1 Statistical Analysis

With few seeds and no known distribution, paired Wilcoxon signed-rank tests were used. All tests are two-sided and uncorrected for multiple comparisons, so they are indicative, and p values near 0.05 are not firm.

Table 7.8: Summary of paired Wilcoxon signed-rank tests

Comparison	Pairs	Result	p
Hypervolume: MOPSO against random search	10	MOPSO higher in all pairs	0.002
Hypervolume: NSGA-II against random search	10	NSGA-II higher in all pairs	0.002
Hypervolume: MOPSO against NSGA-II	10	NSGA-II higher in 9 of 10	0.014
Hypervolume at 25 / 200 iterations: MOPSO against NSGA-II	5	MOPSO ahead at 25, NSGA-II at 200 (all pairs)	0.062 / 0.062
Held-out Sharpe: best-Sharpe picks against equal weight	30	Not different (MOPSO, NSGA-II)	0.428 / 0.416
Held-out volatility: minimum-volatility picks against equal weight	30	Lower in all 30 (both)	< 0.001
Held-out drawdown: NSGA-II minimum-volatility against equal weight	30	Shallower in 24 of 30	0.005
Ablation: with and without turbulence / swap mutation	8	Not different	0.641 / 0.547

7.2 Figures

Figure 7.1 and the next figure show single quantities: front quality against budget, and the training against held-out Sharpe ratio of each strategy.

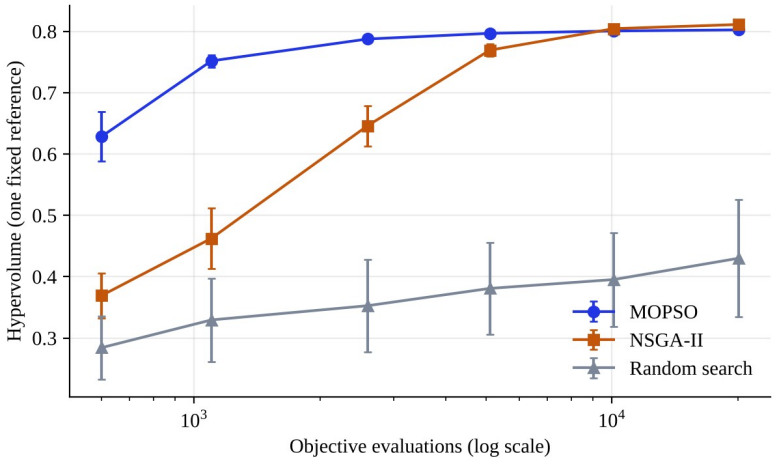


Figure 7.1: Hypervolume against the number of objective evaluations (error bars: standard deviation over five seeds).

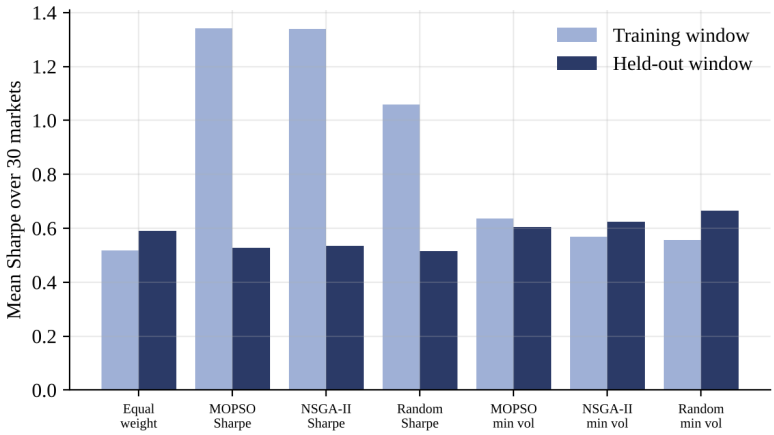


Figure 7.2: Mean Sharpe ratio on the training and the held-out window for each strategy, over 30 markets.

7.3 Subfigures

The multi-panel figures below show the fronts, the convergence and the distributions behind the tables.

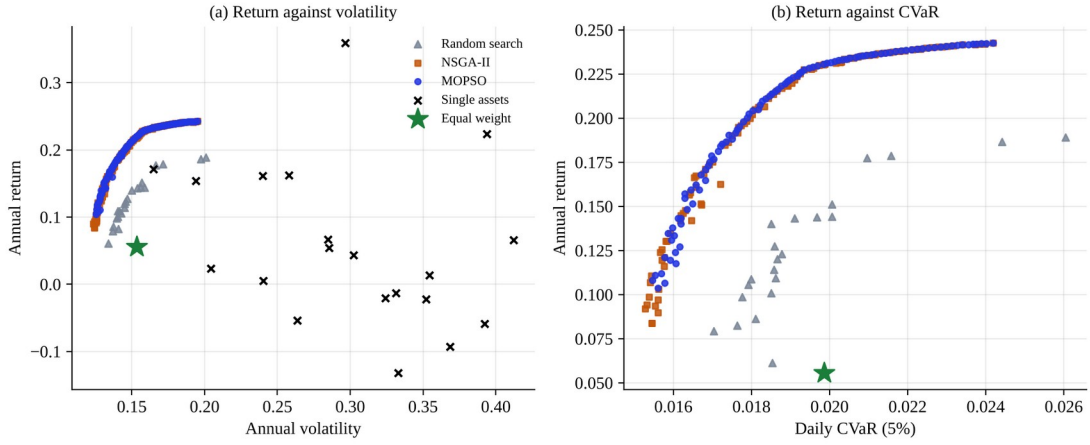


Figure 7.3: Fronts found on the training window of the default market: (a) return against volatility; (b) return against CVaR.

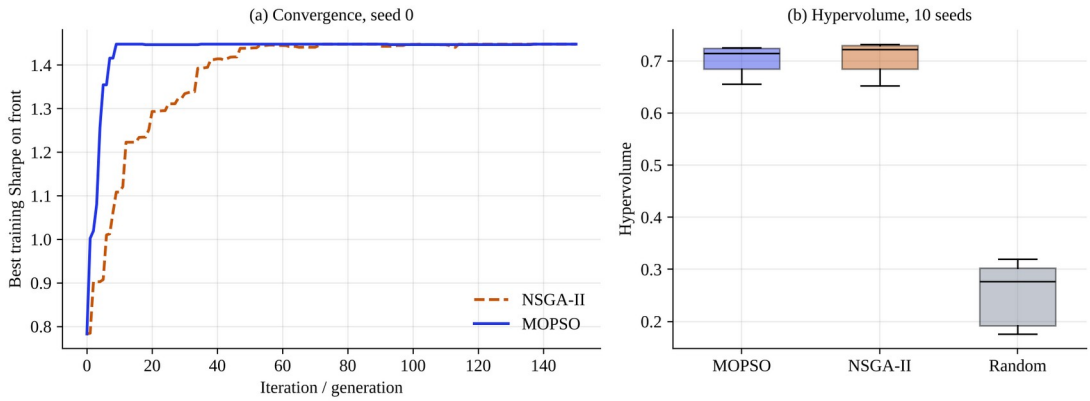


Figure 7.4: Best training Sharpe on the front per iteration for seed 0 (a) and hypervolume over ten seeds (b).

In Figure 7.4(a), MOPSO reaches 99% of its final best Sharpe ratio after 9 iterations and NSGA-II after 47, and both end at 1.448.

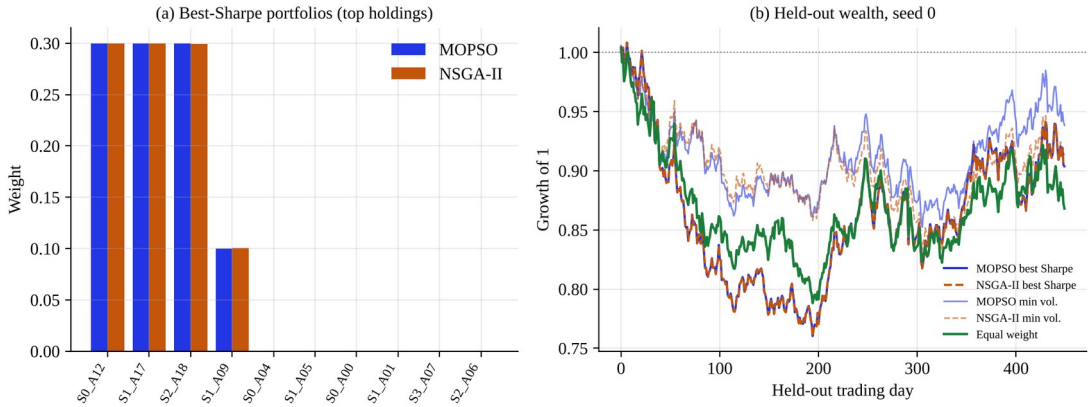


Figure 7.5: Best-Sharpe portfolios on the default market: (a) weights of the largest holdings; (b) held-out wealth against equal weight.

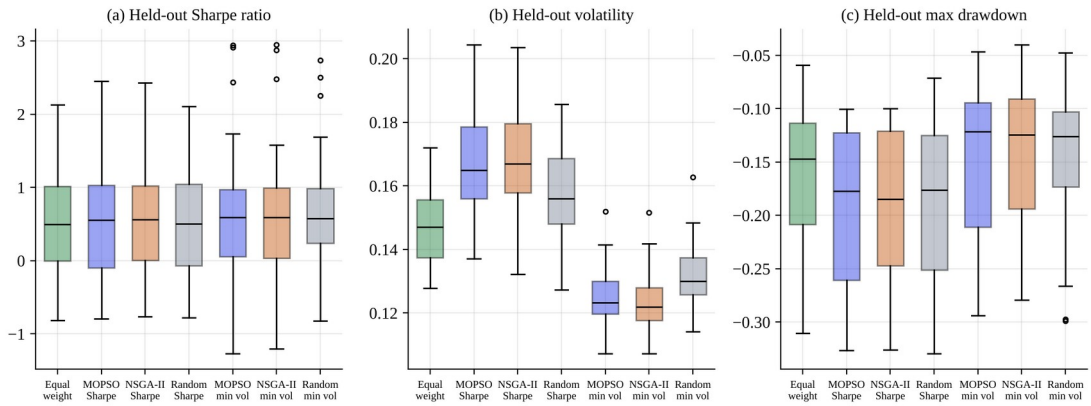


Figure 7.6: Held-out distributions over 30 markets: (a) Sharpe ratio; (b) volatility; (c) maximum drawdown.

7.4 Discussion

Both optimisers approximate the front well and are close to each other. Both reach a hypervolume of about 0.70 against about 0.25 for random search and return about 100 non-dominated portfolios. NSGA-II is marginally ahead at the default budget and MOPSO has

the more even front, but the differences are small. The budget study shows why the ranking depends on the set-up: MOPSO reaches 0.75 after 10 iterations, which NSGA-II needs 25 to 50 iterations to match, whereas NSGA-II keeps improving and finishes slightly ahead from 100 iterations. Neither is better overall: MOPSO is preferable when evaluations are scarce and NSGA-II when they are plentiful.

Constraints and cost. With one repair operator no run violated the cap or the holding limit, and tight limits did not hurt the best Sharpe ratio because the best portfolio holds only four assets. A default run takes about a second and a half, almost independent of the number of assets, and the advantage over random search widens as the universe grows.

Operator ablations show no measurable benefit. Removing the turbulence step or the swap mutation changed the hypervolume by less than 0.002 on average, within seed-to-seed variation. At these settings they are harmless safeguards and not drivers of performance.

A better front does not mean a better portfolio on new data. Portfolios chosen for the best Sharpe ratio looked excellent in training (1.34) and fell to 0.53 on unseen data, below equal weight at 0.59, and random search did about as well. This agrees with Michaud [6] and DeMiguel et al. [7]. A simple calculation shows why: with about four years of data, the standard error of an annual mean return is roughly the annual volatility divided by the square root of the number of years, about 12 percentage points for an asset with 25% volatility, which is as large as the whole spread of true alphas used to generate the markets (-2% to 10%). The optimiser therefore mostly fits noise in the means.

Risk-based choices do transfer. Volatility is estimated far more precisely from the same data. The lowest-volatility portfolios had lower held-out volatility than equal weight in all 30 markets and often shallower drawdowns, with a similar Sharpe ratio. Selecting from the low-risk end of the front is therefore the more dependable choice, and the high-return end should be treated with scepticism. Since random search also finds acceptable minimum-volatility portfolios, the value of the metaheuristics lies in producing a full, well-spread front.

Limitations

- All markets are simulated; real prices have time-varying correlations and regime changes, so the held-out findings must be repeated on real data.
- Hypervolume is estimated by sampling and depends on the reference point; algorithm parameters were fixed at common values and not tuned.

- Two optimisers, zero risk-free rate, long-only portfolios, no trading costs, uncorrected tests and five to ten seeds give limited power.

7.5 Conclusion

Both MOPSO and NSGA-II find much better fronts than random search under identical constraints and budget. MOPSO is faster at small budgets and NSGA-II marginally better at large ones. Portfolios chosen for return do not beat equal weight on unseen data, while low-risk portfolios keep their risk advantage.

Chapter 8

Conclusion

8.1 Conclusion

This project built SwarmFolio, a library and a browser workbench for multi-objective portfolio selection with a particle swarm optimiser and a genetic algorithm. Both optimisers search one shared constrained space of long-only portfolios, trading off annual return, volatility and CVaR, and both are covered by an automated test suite and interface tests in a real browser. The main findings are:

- Both algorithms approximate the Pareto front far better than random search at equal budget (hypervolume about 0.70 against 0.25) and respect every constraint in every run.
- MOPSO converges in far fewer iterations and gives a more even front; NSGA-II ends marginally ahead from 100 iterations. At the default budget they cannot be separated in practical terms.
- A run takes about a second and a half, nearly independent of the number of assets up to 60, and the extra operators did not measurably change front quality.
- Best-Sharpe portfolios did not outperform equal weight on held-out data over 30 simulated markets, while minimum-volatility portfolios kept clearly lower held-out volatility in all 30. Training quality is therefore not a promise of future return.

The limits are stated in Section 7.4: the markets are simulated, parameters were not tuned and the number of seeds is modest.

8.2 Future Scope

- **Real-market validation:** repeat the held-out study on historical prices across many rolling windows, using the existing CSV import.
- **Better estimation:** add shrinkage covariance and robust return estimates, the standard remedy for the estimation-error problem seen here.
- **More objectives and algorithms:** turnover and trading cost as objectives, sector limits, decomposition-based algorithms and tuned parameters.
- **Deployment and teaching:** authentication and an isolated worker so the workbench can be hosted for a class, with guided exercises on dominance and over-fitting.

B.M.S. College of Engineering.

Department of AI & DS

Attainment of POs and PSOs

Batch no.: _____ Date: _____

Project Title: SwarmFolio: Multi-Objective Stock Portfolio Optimization Using Particle Swarm Optimization and Genetic Algorithms

PROGRAM OUTCOMES

PO	Level (3/2/1) 3-High 2-Medium 1-Low	Justification if addressed
PO1	3	Portfolio theory, statistics and multi-objective optimisation were applied to formulate and solve the portfolio problem in software (Chapters 1, 4 and 5).
PO2	3	The problem was formulated with three objectives and four constraints; literature was reviewed and gaps identified (Chapter 2) and conclusions were supported by experiments (Chapter 7).
PO3	2	A layered system (library, server, browser interface) was designed to the requirements of Chapter 3 and built (Chapters 4 and 5), with input validation and security measures (Section 5.4).
PO4	3	Six controlled experiments with seeds, statistical tests and ablations were designed, run and analysed (Chapters 4, 6 and 7).
PO5	3	NumPy, pandas, Matplotlib, SciPy, pytest and a browser canvas interface were used, and their limitations are stated (Sections 3.4, 5.2 and 7.4).
PO9	2	This report, the presentation and the interactive interface communicate the work to technical and non-technical readers (Section 4.1.3).
PO10	2	A work plan, a cost estimate and a risk register were prepared (Sections 1.8, 3.5 and 3.7).

PROGRAM SPECIFIC OUTCOMES

PSO	Level (3/2/1) 3-High 2-Medium 1-Low	Justification if addressed
PSO1	3	A computing system using AI techniques (particle swarm and evolutionary search) was designed and developed (Chapters 4 and 5).
PSO2	2	Optimisation and statistical evaluation were applied to an investment decision problem, and the results were interpreted for decision making (Chapter 7).
PSO3	3	An end-to-end system was built: data generation and import, optimisation, evaluation, an interactive interface and tests (Chapters 4 to 6).

REFERENCES

[1] H. Markowitz, "Portfolio selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77-91, Mar. 1952.

[2] R. T. Rockafellar and S. Uryasev, "Optimization of conditional value-at-risk," *The Journal of Risk*, vol. 2, no. 3, pp. 21-41, 2000.

[3] T.-J. Chang, N. Meade, J. E. Beasley, and Y. M. Sharaiha, "Heuristics for cardinality constrained portfolio optimisation," *Computers & Operations Research*, vol. 27, no. 13, pp. 1271-1302, 2000.

[4] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE Int. Conf. Neural Networks*, vol. 4, Perth, WA, Australia, 1995, pp. 1942-1948.

[5] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182-197, Apr. 2002.

[6] R. O. Michaud, "The Markowitz optimization enigma: Is 'optimized' optimal?" *Financial Analysts Journal*, vol. 45, no. 1, pp. 31-42, Jan.-Feb. 1989.

[7] V. DeMiguel, L. Garlappi, and R. Uppal, "Optimal versus naive diversification: How inefficient is the 1/N portfolio strategy?" *The Review of Financial Studies*, vol. 22, no. 5, pp. 1915-1953, May 2009.

[8] United Nations, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, 2015. [Online]. Available: https://sdgs.un.org/2030agenda. [Accessed: Oct. 30, 2026].

[9] T. Cura, "Particle swarm optimization approach to portfolio optimization," *Nonlinear Analysis: Real World Applications*, vol. 10, no. 4, pp. 2396-2406, Aug. 2009.

[10] H. Zhu, Y. Wang, K. Wang, and Y. Chen, "Particle swarm optimization (PSO) for the constrained portfolio optimization problem," *Expert Systems with Applications*, vol. 38, no. 8, pp. 10161-10169, Aug. 2011.

[11] G.-F. Deng, W.-T. Lin, and C.-C. Lo, "Markowitz-based portfolio selection with cardinality constraints using improved particle swarm optimization," *Expert Systems with Applications*, vol. 39, no. 4, pp. 4558-4566, Mar. 2012.

[12] K. P. Anagnostopoulos and G. Mamanis, "A portfolio optimization model with three objectives and discrete variables," *Computers & Operations Research*, vol. 37, no. 7, pp. 1285-1297, Jul. 2010.

[13] P. Skolpadungket, K. Dahal, and N. Harnpornchai, "Portfolio optimization using multi-objective genetic algorithms," in *Proc. IEEE Congr. Evolutionary Computation*, Singapore, 2007, pp. 516-523.

[14] K. Metaxiotis and K. Liagkouras, "Multiobjective evolutionary algorithms for portfolio management: A comprehensive literature review," *Expert Systems with Applications*, vol. 39, no. 14, pp. 11685-11698, Oct. 2012.

[15] W. F. Sharpe, "Mutual fund performance," *The Journal of Business*, vol. 39, no. 1, pt. 2, pp. 119-138, Jan. 1966.

[16] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI, USA: Univ. Michigan Press, 1975.

[17] K. Deb and R. B. Agrawal, "Simulated binary crossover for continuous search space," *Complex Systems*, vol. 9, no. 2, pp. 115-148, 1995.

[18] Y. Shi and R. Eberhart, "A modified particle swarm optimizer," in *Proc. IEEE Int. Conf. Evolutionary Computation*, Anchorage, AK, USA, 1998, pp. 69-73.

[19] C. A. Coello Coello, G. T. Pulido, and M. S. Lechuga, "Handling multiple objectives with particle swarm optimization," *IEEE Trans. Evol. Comput.*, vol. 8, no. 3, pp. 256-279, Jun. 2004.

[20] E. Zitzler, L. Thiele, M. Laumanns, C. M. Fonseca, and V. Grunert da Fonseca, "Performance assessment of multiobjective optimizers: An analysis and review," *IEEE Trans. Evol. Comput.*, vol. 7, no. 2, pp. 117-132, Apr. 2003.

[21] E. Zitzler and L. Thiele, "Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach," *IEEE Trans. Evol. Comput.*, vol. 3, no. 4, pp. 257-271, Nov. 1999.

[22] J. R. Schott, "Fault tolerant design using single and multicriteria genetic algorithm optimization," M.S. thesis, Dept. Aeronaut. Astronaut., Massachusetts Inst. Technol., Cambridge, MA, USA, 1995.

[23] K. Deb, *Multi-Objective Optimization Using Evolutionary Algorithms*. Chichester, U.K.: Wiley, 2001.

[24] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357-362, Sep. 2020.

[25] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf.*, Austin, TX, USA, 2010, pp. 56-61.

[26] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90-95, May-Jun. 2007.

[27] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261-272, Mar. 2020.

[28] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80-83, Dec. 1945.